

Question 1

Steps:

1. For each index
 If $\text{current}[i] \neq \text{goal}[i]$
 Cost++
2. It lower bounds the min number of one letter replacement actions needed to reach the goal.
3. It cannot overestimate because actions change one letter at a time meaning the min number of actions is the min number of letter replacements and this heuristic assumes there are no intermediate dictionary words, representing the best case scenario.

Scrabble:

1. For each index
 If $\text{current}[i] \neq \text{goal}[i]$
 Cost += scrabble_cost(char at i)
2. The min accumulated scrabble cost of the mismatched letters.
3. To reach the goal we must eventually insert every mismatched letter, this heuristic doesn't take into account intermediate dictionary words thus ignoring any real additional costs.

Frequency:

1. For each index
 If $\text{current}[i] \neq \text{goal}[i]$
 Cost++
2. The min accumulated frequency cost of the mismatched letter.
3. The cost of action to produce a word is $1 + \text{rarity}(\text{word})$, because rarity is nonnegative the cheapest cost is atleast 1. Fixing N mismatched letters results in atleast N actions therefore N+ cost.

Question 2

Word pair	Cost measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
WHY → ASK	Steps	5	773	5	16	97.93%
WHY → ASK	Scrabble	9	765	9	13	98.30%
WHY → ASK	Frequency	38.86	808	38.86	755	6.56%
BEAR → DUCK	Steps	4	691	4	6	99.13%
BEAR → DUCK	Scrabble	11	2981	11	6	99.80%
BEAR → DUCK	Frequency	38.24	1084	38.24	720	33.58%

1. They always returned the same costs which confirms the heuristic for A* is admissible.
2. Scrabble produced the largest reduction but steps was a close second.
3. For both word pair, frequency was the least informative.

4. Scrabble is highly informative because it calculates the actual target letter values which scale closely with the true path cost. Frequency is the least because it only counts the mismatched letters. Since the true frequency cost is $1 + \text{rarity}$ and is usually much higher the, heuristic underestimates remaining cost making A* behave like UCS.

Question 3

Steps:

1. Yes
2. Every action has cost of 1, and are reversible. Traversal in either way will have the same cost since it's just the number of steps.

Scrabble:

1. No
2. Each letter has a different cost, so for $W1 \rightarrow W2$ we pay for $W2$ chars, but from $W2 \rightarrow W1$ we pay for $W1$ chars. Different letters have different cost so the path is asymmetrical.
3. Dog $\rightarrow$ cog costs 3 but cog $\rightarrow$ dog costs 2.

Frequency:

1. No
2. Similar to scrabble, the paths are asymmetric.
3. Dog $\rightarrow$ cog costs $1 + \text{rarity}(\text{cog})$ but cog $\rightarrow$ dog costs $1 + \text{rarity}(\text{dog})$

Question 4

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

Show more ▾

Worked for 7s

A simple rarity-aware admissible heuristic is to account for the rarity cost of the goal word, since every solution must eventually enter the goal.

Let

$$h_0(n) = \text{HammingDistance}(n, g)$$

and let $r(g)$ be the rarity of the goal word g . Define

$$h_R(n) = \begin{cases} 0, & n = g \\ h_0(n) + r(g), & n \neq g \end{cases}$$

GPT-5.6 Luna

3. Yes it's admissible, any valid path requires atleast $h_0(n)$ actions with base cost 1, establishing a min base cost of $h_0(n)$. Also because we get to the goal we have to pay for the goal. By adding min base cost and goal cost while ignoring the intermediate costs, the heuristic calculates best case never overestimating.

5. The heuristic showed no improvement whatsoever. The solution cost remained optimal (38.86 and 38.24) confirming its admissible. However the number of expanded nodes from baseline was identical. In conclusion, the evidence doesnt support the proposed heuristic, it failed to understand how A* actually evaluates nodes. Luna simply adds a constant value to all non goal states. Adding a constant to every node doesnt change their relative priority in the A* queue, causing the search to explore the same path.